

Compiler Construction

~ Instruction Selection ~

Problem Statement

How would you translate $\mathbf{a}[\mathbf{i}] := \mathbf{x}$ where $\mathbf{a}$ and $\mathbf{x}$ are frame resident, and $\mathbf{i}$ is not?

Problem Statement

How would you translate $\mathbf{a}[\mathbf{i}] := \mathbf{x}$ where $\mathbf{a}$ and $\mathbf{x}$ are frame resident, and $\mathbf{i}$ is not?

- $\mathbf{i}$ can be stored in a temporary

Problem Statement

How would you translate $\mathbf{a}[\mathbf{i}] := \mathbf{x}$ where $\mathbf{a}$ and $\mathbf{x}$ are frame resident, and $\mathbf{i}$ is not?

- $\mathbf{i}$ can be stored in a temporary
- $\mathbf{a}$ and $\mathbf{x}$ can be referred to relatively to $\$fp$

Problem Statement

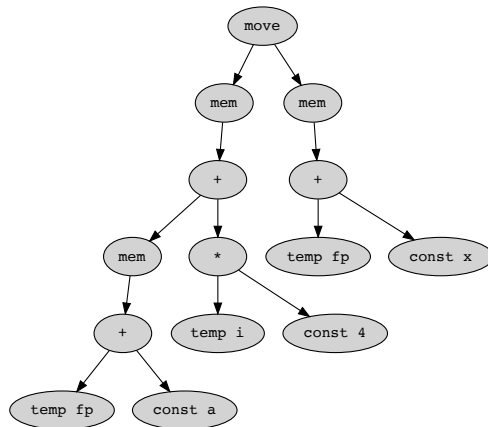
How would you translate $\mathbf{a[i] := x}$ where $\mathbf{a}$ and $\mathbf{x}$ are frame resident, and $\mathbf{i}$ is not?

- $\mathbf{i}$ can be stored in a temporary
- $\mathbf{a}$ and $\mathbf{x}$ can be referred to relatively to $\$fp$
- $\mathbf{a[i]}$ is a memory access at an offset of $\mathbf{i}$ from the start of $\mathbf{a}$

Problem Statement

How would you translate $\mathbf{a[i] := x}$ where $\mathbf{a}$ and $\mathbf{x}$ are frame resident, and $\mathbf{i}$ is not?

- $\mathbf{i}$ can be stored in a temporary
- $\mathbf{a}$ and $\mathbf{x}$ can be referred to relatively to $\$fp$
- $\mathbf{a[i]}$ is a memory access at an offset of $\mathbf{i}$ from the start of $\mathbf{a}$



From $a[i] := x$ to Assembly

One *possible* translation is :

```
load  t17 <- M[fp + a]
addi  t18 <- r0 + 4
mul   t19 <- ti * t18
add   t20 <- t17 + t19
load  t21 <- M[fp + x]
store M[t20+0] <- t21
```

How did we compute it?

Instruction Selection

The Core Idea

Instead of deciding *how to translate* each IR construct (Move, Cjump, Call, ...) into assembly step by step, we define:

Which IR trees can be covered by which machine instructions

- Machine instructions are viewed as **tree patterns (tiles)**
- Code generation becomes a **tree covering problem**
- Each tile:
 - ▶ Matches a specific IR shape
 - ▶ Produces assembly
 - ▶ Has a cost

Pros/Cons

Pros

- Clean separation between IR and architecture
- Systematic and automatable
- Enables cost-based optimization
- Naturally supports complex instructions

Pros/Cons

Pros

- Clean separation between IR and architecture
- Systematic and automatable
- Enables cost-based optimization
- Naturally supports complex instructions

Cons

- Requires carefully designed tile set
- Cost model may not reflect real CPU behavior
- Ignores instruction-level parallelism and pipeline effects

Two Possible Approaches

Ad-hoc approach

- For each IR node (Move, Cjump, Call, ...)
- Manually decide which sequence of assembly instructions to emit
- Simple to start
- Quickly becomes complex and unmaintainable
- Many special cases
- Hard to optimize globally

Tile-based approach

- Define which IR trees can be covered by each instruction
- Let an algorithm select the best covering
- Harder to design initially
- Requires defining tiles + cost model
- Once in place: systematic and extensible
- Adding instructions = adding tiles

Tree Patterns A.K.A Tiles

- Translation from Tree to Assembly corresponds to *parsing a tree*.
- Looking for a covering of the tree, using tiles.
- The set of tiles corresponds to the instruction set.

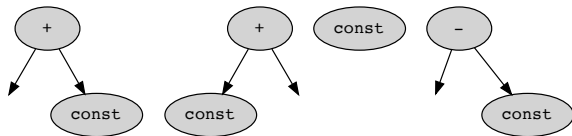


Tiles

`addi $rt, $rs, 0` is addition between a register and some immediate ...

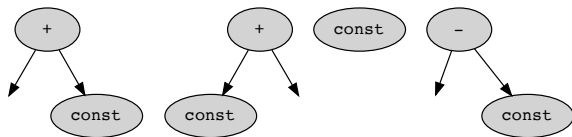
Tiles

`addi $rt, $rs, 0` is addition between a register and some immediate ... but also on some immediate and a register :



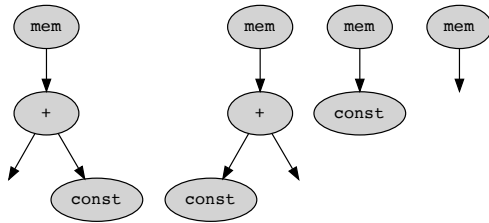
Tiles

`addi $rt, $rs, 0` is addition between a register and some immediate ... but also on some immediate and a register :

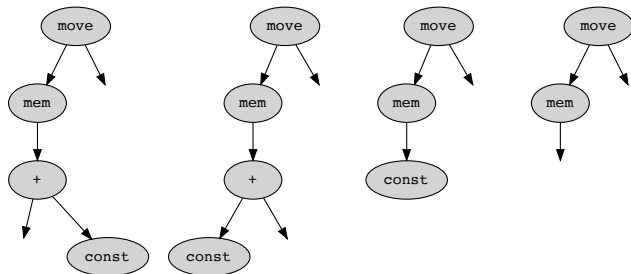


Missing nodes are plugs for *temporaries*: tiles read from temps, and create temps.

Tiles: Loading load $r_i \leftarrow M[r_j + c]$

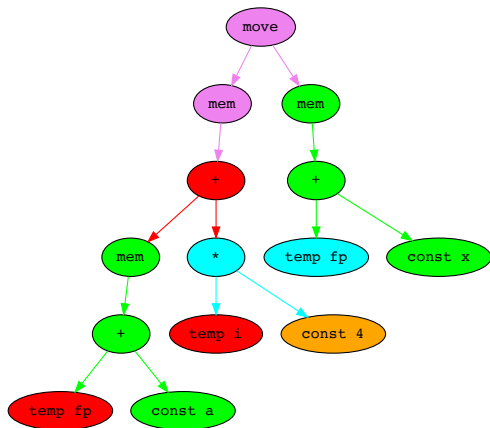


Tiles: Storing $\text{store } M[r_j + c] \leftarrow r_i$



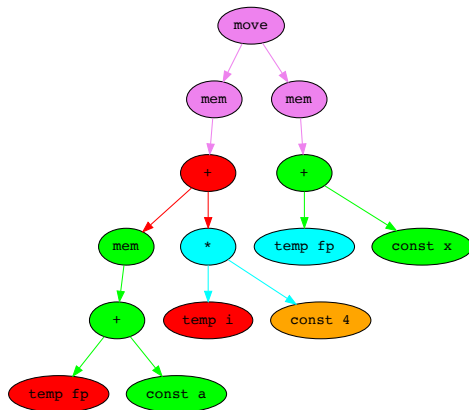
Simple Instruction: Translation 1

```
load t17 <- M[fp + a]
addi t18 <- r0 + 4
mul t19 <- ti * t18
add t20 <- t17 + t19
load t21 <- M[fp + x]
store M[t20+0] <- t21
```



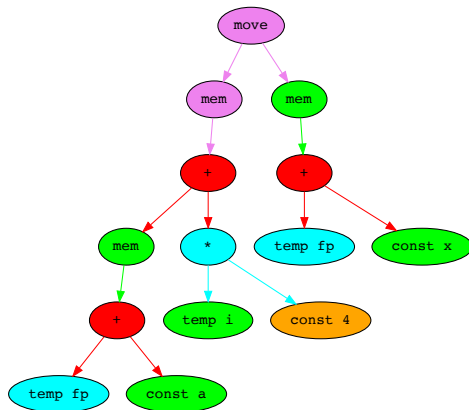
Simple Instruction: Translation 2

```
load t17 <- M[fp + a]
addi t18 <- r0 + 4
mul t19 <- ti * t18
add t20 <- t17 + t19
addi t21 <- fp + x
movem M[t20] <- M[t21]
```



Simple Instruction: Translation 3

```
addi t17 <- r0 + a
add t18 <- fp + t17
load t19 <- M[t18 + 0]
addi t20 <- r0 + 4
mul t21 <- ti * t20
add t22 <- t19 + t21
addi t23 <- r0 + x
add t24 <- fp + t23
load t25 <- M[t24 + 0]
store M[t22 + 0] <- t25
```



Translating a Simple Instruction

- There is always a solution
(provided the instruction set is reasonable)
- there can be several solutions
- given a cost function, some are better than others:
 - ▶ some are locally better, *optimal coverings*
(no fusion can reduce the cost),
 - ▶ some are globally better, *optimum coverings*.

Translating a Simple Instruction

- There is always a solution
(provided the instruction set is reasonable)
- there can be several solutions
- given a cost function, some are better than others:
 - ▶ some are locally better, *optimal coverings*
(no fusion can reduce the cost),
 - ▶ some are globally better, *optimum coverings*.

Nowadays

This approach is too naive: CPUs are really layers of units that work in parallel → Costs are interrelated.

Algorithms for Instruction Selection (1/2)

Maximal Munch Find an optimal tiling.

- Top-down strategy.
- Cover the current node with the largest tile.
- Repeat on subtrees.
- Generate instructions in reverse-order after tile placement.

Algorithms for Instruction Selection (2/2)

Dynamic Programming Find an optimum tiling.

- Bottom-up strategy.
- Assign cost to each node.
- Cost = cost of selected tile + cost of subtrees.
- Select a tile with minimal cost and recur upward.
- Implemented by code generator generators (Twig, Burg, iBurg, MonoBURG, ...).

Instruction Selection in OCaml

From **Tree**.program (lir) to **Asm**.program

Instruction Selection in OCaml

From **Tree**.program (lir) to **Asm**.program

Using Tiles

The Asm Module (Preview)

- An intermediate **registerless** representation of RISC-V assembly
- Structured (not raw strings)
- Easier to manipulate than text

Example constructors:

```
1 Asm.binary_instr ~op:Add ~dst:t ~src1:ta ~src2:tb  
2 Asm.move_instr ~dst:t ~src:s  
3 Asm.branch rel t1 t2 ~ltrue:l1 ~lfalse:l2
```

Important:

- Tiles produce `Asm.instr list`
- Final pass pretty-prints to real RISC-V

We will revisit Asm in detail later.

Instruction Selection in OCaml

- Each instruction pattern is represented as a `tile`.
- A tile describes:
 - ▶ **Cost** of using this instruction
 - ▶ A `matches_exp` function to see if the tile applies
 - ▶ An `emit_exp` function to generate RISC-V code
- Bottom-up tree traversal selects the cheapest matching tile for each node.

```
1 type tile = {  
2   cost : int;  
3   matches_exp : Tree.expr -> bool;  
4   emit_exp : (Tree.expr -> Asm.temp) -> Tree.expr  
5             -> string Utils.SMap.t  
6             -> Asm.instr list * Asm.temp;  
7 }
```

Pattern Matching: `matches_exp`

Purpose: Determine whether this tile applies to a given IR node.

Example: match any binary operation

```
1 matches_exp = function  
2   | { payload = Binop op e1 e2 } -> true  
3   | any -> false
```

Important:

- No code generation here
- Only structural inspection
- Cheap test

Understanding emit_exp

Signature:

```
1 emit_exp :  
2   (Tree.expr -> Asm.temp) ->  
3   Tree.expr ->  
4   string Utils.SMap.t ->  
5   Asm.instr list * Asm.temp
```

- (Tree.expr -> Asm.temp) helper to recursively lower sub-expressions
- Tree.expr the expression this tile is responsible for
- string Utils.SMap.t mapping of float literals for assembly emission
- Asm.instr list * Asm.temp the instructions generated and the temporary holding the result

What Does `emit_exp` Do?

When a tile is selected, it must:

- Recursively lower child expressions
- Allocate a fresh temporary
- Generate RISC-V instruction(s)
- Return:
 - ▶ Instruction list
 - ▶ Result temporary

It is responsible for **actual code generation**.

Emission Example: Binary Add

```
1 | Binop(op,a,b) ->
2 |   let ta = expr_to_temp a in
3 |   let tb = expr_to_temp b in
4 |   let t = r.fresh_temp Int in
5 |   ([ Asm.binary_instr
6 |     ~op:(asm_of_binop op)
7 |     ~dst:t
8 |     ~src1:ta
9 |     ~src2:tb ], t)
```

Step by step:

- 1 Lower left operand `ta`, Lower right operand `tb`
- 2 Allocate new temp `t`
- 3 Emit RISC-V instruction
- 4 Return (instructions, `t`)

Why Return a Temporary?

Expression lowering is compositional.

Example:

$$(a + b) * c$$

- Add tile returns temp t_1
- Multiply tile uses t_1 as input

Each tile produces a value in a temporary, so parents can use it.

This builds code bottom-up.

Full Add tile

```
1 let tile_binop r = {  
2   cost = 1;  
3   matches_exp = function { payload = Binop o } -> true | any -> false;  
4   emit_exp = fun expr_to_temp e maps ->  
5     match e.payload with  
6     | Binop(op,a,b) ->  
7       let ta = expr_to_temp a in  
8       let tb = expr_to_temp b in  
9       let t = r.fresh_temp Int in  
10      ([ Asm.binary_instr ~op:(asm_of_binop op) ~dst:t  
11        ~src1:ta ~src2:tb ], t)  
12    | any -> assert false  
13 }
```

emit_exp is only called if the expression matches, hence the assert **false**

Lowering Statements

- Statements are translated via `munch_stm`
- Calls `munch_exp` for expressions in `move`, `jump`, or `cjump`

```
1 let munch_stm r float_literals s =  
2   match s.payload with  
3   | Move(t,e) -> [ Asm.move_instr ~dst:t ~src:(munch_exp r e) ]  
4   | Cjump(rel,a,b,l1,l2) ->  
5     [ Asm.branch (asm_of_rel rel) (munch_exp r a) (munch_exp r b) ~ltrue  
6   | Jump(Name l) -> [ Asm.jump l ]  
7   | Label l -> [ Asm.label l ]  
8   | any -> failwith "not supported"
```

Key Ideas Behind OCaml Implementation

- **Bottom-up traversal** ensures subexpressions are computed first
- **Tiles encode both pattern matching and code emission**
- **Temporaries** are used to store intermediate results
- **Cost model** lets the selector pick the "best" instruction
- Works for constants, binary ops, calls, memory access, etc.

Instruction Selection Flow (Simplified)

- **IR Program**
- Recursively process expressions via `munch_exp`
 - ▶ Match IR nodes against tiles
 - ▶ Generate instructions into temporary registers
- Process statements via `munch_stm`
 - ▶ Handle moves, jumps, labels, calls
 - ▶ Use `expr_to_temp` to evaluate sub-expressions
- Emit RISC-V assembly instructions
 - ▶ Tile-based code generation
 - ▶ Instruction buffer collected and output
- Result: complete RISC-V assembly

Functorized Architecture Backend

- Instruction selection engine is parameterized by a target backend
- Functor `Make(A : ARCH)` builds a selector for any architecture
- `ARCH` signature defines:
 - ▶ Tiles for instruction selection (`tiles`)
 - ▶ Statement lowering (`munch_stm`)
 - ▶ Registers and calling convention (`registers`, `caller_saved`, `callee_saved`)
 - ▶ File prologue/epilogue
- This allows the same code to generate RISC-V, ARM, etc.

Functorized Architecture Backend

```
1 module type ARCH = sig
2   val tiles : Tree_helper.rewriter -> tile list
3   val munch_stm : Tree_helper.rewriter -> (Tree.expr -> Asm.temp) ->
4       Tree.stmt -> Asm.instr list
5   val registers : string list
6   val callee_saved : string list
7   val caller_saved : string list
8   val file_prologue : string
9   val file_epilogue : string
10 end
```

Summary

